Set_A_Lab7 - Results

Attempt 1 of 1

Written Mar 4, 2026 11:35 AM - Mar 4, 2026 12:11 PM

Attempt Score	**14 / 14 - 100 %**
Overall Grade (Highest Attempt)	**14 / 14 - 100 %**

Question 1

Consider the following C code, use the online C compiler

https://www.onlinegdb.com/online_c_compiler

to run your code.

```c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

#define NUM_THREADS 3
#define INCREMENTS 100000

int counter = 0;

void* increment_counter(void* arg) {
    for (int i = 0; i < INCREMENTS; i++) {
        counter++;  // Critical section
    }
    return NULL;
}

int main() {
    pthread_t threads[NUM_THREADS];

    for (int i = 0; i < NUM_THREADS; i++) {
        if (pthread_create(&threads[i], NULL, increment_counter, NULL) != 0) {
            perror("Failed to create thread");
            return 1;
        }
    }
    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_join(threads[i], NULL);
    }
    printf("Final counter value: %d\n", counter);
    return 0;
}
```

Insert a screenshot of your output.

main.c

```c
void* increment_counter(void* arg) {
    for (int i = 0; i < INCREMENTS; i++) {
        counter++; // Critical Section
    }
    return NULL;
}

int main() {
    pthread_t threads[NUM_THREADS];

    for (int i = 0; i < NUM_THREADS; i++) {
        if (pthread_create(&threads[i], NULL, increment_counter, NUL
            perror("Failed to create thread");
            return 1;
        }
    }

    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_join(threads[i], NULL);
    }

    printf("Final counter value: %d\n", counter);
    return 0;
}
```

input

```
Final counter value: 227259


...Program finished with exit code 0
Press ENTER to exit console.
```

Screenshot 2026-03-04 at 11.40.42 AM.png (247.87 KB)

The correct answer is not displayed for Written Response type questions.

Question 2

Referring to your results from Question 1,

What is the final value of the counter without a race condition? e.g. 12345

___300000___ ✔(50 %)

Run the program multiple times. Do you obtain the same value on each execution (Y/N)? ___N___ ✔(50 %)

Question 3

Consider the following C code (https://cocalc.com/), first compile your code to obtain the binary file *lock.out*.

gcc lock.c -pthread -o lock.out

lock.c

```c
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

// int lock = 0;
int sum = 0;

void criticalRegion(){
        for(int i=0; i<10000; i++){
            sum += 1;
        }
    }

void *p(void *arg){
    // while(lock!=0);
    // lock = 1;
    criticalRegion();
    // lock = 0;
}

int main(int argc, char *argv[])
{
    int status0, status1;
    pthread_t threadP0, threadP1;
    status0 = pthread_create(&threadP0, NULL, p, NULL);
    status1 = pthread_create(&threadP1, NULL, p, NULL);
    pthread_join(threadP0, NULL);
    pthread_join(threadP1, NULL);
    printf("Sum = %d\t", sum);
    return 0;
}
```

In order to execute lock.out 400 times you need to prepare a bash script file (called run) as follows:

runLock.sh

```
#!/bin/bash
clear
for i in {1..400}
do
    ./lock.out
    if (( $i % 5 == 0))
    then
        printf "\n"
    fi
done
```

Then use the “chmod +x” command so that the system recognizes it as an executable file.

chmod +x runLock.sh

Now run your bash script (*./runLock.sh*) and answer the following questions:

Insert a screenshot of your output.

2026-03-04-file-1.term

Assistant Server ~/lab7 Terminal File Edit View Go Help Chat

```
Sum = 16987     Sum = 17530     Sum = 20000     Sum = 20000     Sum = 20000
Sum = 15238     Sum = 20000     Sum = 20000     Sum = 20000     Sum = 17438
Sum = 20000     Sum = 20000     Sum = 16679     Sum = 12251     Sum = 20000
Sum = 20000     Sum = 17630     Sum = 20000     Sum = 20000     Sum = 16344
Sum = 17030     Sum = 20000     Sum = 17217     Sum = 20000     Sum = 20000
Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000
Sum = 20000     Sum = 20000     Sum = 17850     Sum = 20000     Sum = 20000
Sum = 20000     Sum = 10144     Sum = 20000     Sum = 18652     Sum = 16163
Sum = 20000     Sum = 17874     Sum = 20000     Sum = 20000     Sum = 16694
Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000
Sum = 20000     Sum = 20000     Sum = 20000     Sum = 12770     Sum = 20000
Sum = 20000     Sum = 13073     Sum = 20000     Sum = 16993     Sum = 20000
Sum = 18281     Sum = 11368     Sum = 20000     Sum = 20000     Sum = 20000
Sum = 20000     Sum = 16055     Sum = 16011     Sum = 18282     Sum = 20000
Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000     Sum = 10708
Sum = 17128     Sum = 10567     Sum = 20000     Sum = 17701     Sum = 20000
Sum = 20000     Sum = 14982     Sum = 19770     Sum = 14430     Sum = 19967
Sum = 20000     Sum = 15868     Sum = 20000     Sum = 11596     Sum = 20000
Sum = 20000     Sum = 20000     Sum = 20000     Sum = 18765     Sum = 20000
Sum = 20000     Sum = 20000     Sum = 20000     Sum = 16547     Sum = 20000
Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000
Sum = 20000     Sum = 18017     Sum = 20000     Sum = 20000     Sum = 14528
Sum = 19442     Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000
Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000
Sum = 15648     Sum = 20000     Sum = 15719     Sum = 20000     Sum = 20000
Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000
Sum = 12750     Sum = 20000     Sum = 13707     Sum = 20000     Sum = 14545
Sum = 10885     Sum = 20000     Sum = 15021     Sum = 17244     Sum = 20000
Sum = 20000     Sum = 20000     Sum = 16599     Sum = 11120     Sum = 17458
Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000
Sum = 20000     Sum = 20000     Sum = 10808     Sum = 20000     Sum = 20000
Sum = 18489     Sum = 15403     Sum = 18911     Sum = 20000     Sum = 20000
Sum = 15267     Sum = 20000     Sum = 20000     Sum = 18485     Sum = 20000
Sum = 15047     Sum = 15803     Sum = 17434     Sum = 17727     Sum = 17803
Sum = 16942     Sum = 12731     Sum = 20000     Sum = 20000     Sum = 13388
Sum = 10406     Sum = 11131     Sum = 20000     Sum = 20000     Sum = 17762
Sum = 20000     Sum = 16747     Sum = 16514     Sum = 15569     Sum = 16239
Sum = 16550     Sum = 12772     Sum = 17289     Sum = 16347     Sum = 16455
Sum = 20000     Sum = 16413     Sum = 14252     Sum = 17671     Sum = 20000
Sum = 20000     Sum = 20000     Sum = 20000     Sum = 11103     Sum = 20000
Sum = 18569     Sum = 18024     Sum = 18953     Sum = 20000     Sum = 20000
Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000
Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000
Sum = 20000     Sum = 20000     Sum = 20000     Sum = 18725     Sum = 20000
Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000
Sum = 20000     Sum = 18977     Sum = 20000     Sum = 20000     Sum = 20000
Sum = 20000     Sum = 18109     Sum = 20000     Sum = 17416     Sum = 20000
Sum = 20000     Sum = 20000     Sum = 15915     Sum = 20000     Sum = 20000
Sum = 10878     Sum = 20000     Sum = 19118     Sum = 20000     Sum = 20000
Sum = 16972     Sum = 19986     Sum = 17221     Sum = 20000     Sum = 15670
Sum = 20000     Sum = 17317     Sum = 11812     Sum = 10593     Sum = 11330
Sum = 10864     Sum = 20000     Sum = 20000     Sum = 16738     Sum = 19010
~/lab7$
```

Screenshot 2026-03-04 at 11.49.49 AM.png (281.23 KB)

The correct answer is not displayed for Written Response type questions.

Question 4

Referring to your results from Question 3,

Does the value of sum remain constant (Y/N)? ___N___ ✔(50 %)

Now uncomment lines 6, 16, 17,and 19 from the file *lock.c*, compile your new code and run the bash script again (*./runLock.sh*).

Does the value of *sum* remain constant this time or not (Y/N)? ___N___ ✔(50 %)

Hint: change {1..400} to {1..800} and run the script many times.

Question 5

Referring to your results from Question 4, insert a screenshot of your output.

Assistant Server ~/lab7 Terminal File Edit View Go Help

```
Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000
Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000
Sum = 20000     Sum = 20000     Sum = 20000     Sum = 14606     Sum = 20000
Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000
Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000
Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000
Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000
Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000
Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000
Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000
Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000
Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000
Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000
Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000
Sum = 20000     Sum = 15043     Sum = 20000     Sum = 20000     Sum = 15093
Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000
Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000
Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000
Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000
Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000
Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000
Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000
Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000
Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000
Sum = 20000     Sum = 20000     Sum = 14555     Sum = 20000     Sum = 20000
Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000
Sum = 20000     Sum = 16412     Sum = 20000     Sum = 20000     Sum = 20000
Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000
Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000
Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000
Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000
Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000
Sum = 20000     Sum = 20000     Sum = 11528     Sum = 20000     Sum = 20000
Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000
Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000
Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000
Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000
Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000
Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000
Sum = 15728     Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000
Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000
Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000
Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000
Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000
Sum = 20000     Sum = 15609     Sum = 20000     Sum = 20000     Sum = 20000
Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000
Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000
Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000
Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000
Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000
Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000     Sum = 20000
~/lab7$
```

Screenshot 2026-03-04 at 11.56.40 AM.png (144.06 KB)

The correct answer is not displayed for Written Response type questions.

Question 6

Repeat question 3 with the following C code (https://cocalc.com/).

gcc peterson.c -pthread -o peterson.out

peterson.c

```c
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

#define FALSE 0
#define TRUE  1
#define N     2

int turn = 0;
int interested[N];
int sum =0;

void do_sum(){
    for(int i=0; i<10000; i++){
        sum += 1;
    }
}

void enter_region(int process){
    int other;
    other = 1- process;
    interested[process] = TRUE;
    turn = process;
    while((turn== process) &&(interested[other]==TRUE));
    do_sum();
}

void leave_region(int process){
    interested[process] = FALSE;
}

void *p(void *arg){
    int *p = (int *)arg;
    enter_region(*p);
    leave_region(*p);
}

int main(int argc, char *argv[])
{
    int status0, status1;
    pthread_t threadP0, threadP1;
    int id0 = 0, id1 = 1;
    status0 = pthread_create(&threadP0, NULL, p, &id0);
    status1 = pthread_create(&threadP1, NULL, p, &id1);
```

```
    pthread_join(threadP0, NULL);
    pthread_join(threadP1, NULL);
    printf("%d", sum);
    return 0;
}
```

runPeter.sh

```
#!/bin/bash

while true
do
  outputA=$(bash -c '(./peterson.out)' 2>&1)
  if [ $outputA -ne 20000 ]; then
    #clear
    echo $outputA
    break
fi
done
```

Insert a screenshot of your output.

```
~/lab7$ chmod +x runPeter.sh
~/lab7$ ./runPeter.sh
15254
~/lab7$
```

Screenshot 2026-03-04 at 12.05.29 PM.png (15.57 KB)

The correct answer is not displayed for Written Response type questions.

Question 7

Referring to your results from Question 6, What is the final value of the sum without a race condition? e.g. 12345? ___20000___ ✔

Question 8

Referring to your results from Question 6, does the value of *sum* remain constant or not, (Y/N)? ___N___ ✔

Question 9

Consider the following C code, use the online C compiler

https://www.onlinegdb.com/online_c_compiler

to run your code.

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <sys/types.h>
#include <sys/wait.h>
int main() {
    int fd[2];
    pid_t pid;
    char message[] = "Operating systems use inter process communication";
    if (pipe(fd) == -1) {
        perror("pipe failed");
        exit(1);
    }
    pid = fork();
    if (pid < 0) {
        perror("fork failed");
        exit(1);
    }
    if (pid == 0) {
        close(fd[1]);
        dup2(fd[0], STDIN_FILENO);
        close(fd[0]);
        execlp("wc", "wc", "-w", NULL);
        perror("execlp failed");
        exit(1);
    } else {
        close(fd[0]);
        write(fd[1], message, strlen(message));
        close(fd[1]);
        wait(NULL);
        printf("Parent: Child finished execution.\n");
    }
    return 0;
}
```

Insert a screenshot of your output.

main.c

```c

    pid = fork();
    if (pid < 0) {
        perror("fork failed");
        exit(1);
    }

    if (pid == 0) {
        close(fd[1]);
        dup2(fd[0], STDIN_FILENO);
        close(fd[0]);
        execlp("wc", "wc", "-w", NULL);
        perror("execlp failed");
        exit(1);
    } else {
        close(fd[0]);
        write(fd[1], message, strlen(message));
        close(fd[1]);
        wait(NULL);
        printf("Parent: Child finished execution.\n");
    }

    return 0;
}
```

input

```
6
Parent: Child finished execution.


...Program finished with exit code 0
Press ENTER to exit console.
```

Screenshot 2026-03-04 at 12.09.09 PM.png (220.2 KB)

The correct answer is not displayed for Written Response type questions.

Question 10

Referring to your results from Question 9, if the parent does NOT **close(fd[1])**, what will most likely happen?

- ○ Child prints 6
- ✔ ◉ Child blocks indefinitely
- ○ Child prints 0
- ○ Parent terminates abnormally

Question 11

Referring to your results from Question 9, what is the main purpose of **dup2(fd[0], STDIN_FILENO)**?

- ○ Copies data from pipe to buffer
- ○ Creates a new pipe
- ✔ ◉ Replaces standard input with pipe read end
- ○ Synchronizes parent and child

Question 12

Referring to your results from Question 9, after **execlp("wc", "wc", "-w", NULL);**, which statement is true?

- ○ Child continues executing the same program
- ○ Code after execlp always executes

- [x] The child process image is replaced
- [ ] A new process is created

Question 13

Referring to your results from Question 9, if we remove **wait(NULL);** in the parent, what could happen?

- [ ] Deadlock
- [ ] Data corruption
- [ ] Pipe failure
- [x] Zombie process

Done